Integração de Sistemas de Informação

Licenciatura em Engenharia de Sistemas Informáticos

TechStore

Integração e Análise de Dados de Vendas de uma Loja de Informática

Trabalho Prático I - Relatório

André Daniel Freitas Ferreira, n.º 20764

UPCA - Universidade Politécnica do Cávado e do Ave

Ano letivo 2026/2027 - outubro de 2026

Índice

1. Enquadramento 3
2. Problema e objetivos 3
3. Estratégia utilizada 3
3.1 Tecnologias utilizadas 4
3.2 Fontes de dados 4
4. Transformações: processo ETL em Python 5
4.1 Extração 6
4.2 Limpeza, normalização e validação 6
4.3 Integração e cálculos 6
4.4 Carregamento 6
5. Estrutura da base de dados PostgreSQL 7
6. Sistema de alertas 7
6.1 Regras de negócio 8
6.2 Funções SQL 8
7. Jobs e orquestração 9
7.1 Execução do ETL 9
7.2 Carregamento no PostgreSQL 9
7.3 Verificação dos alertas 9
7.4 Agendamento no Windows (preparado, não instalado) 10
8. Dashboard Streamlit 10
9. Testes e resultados 12
10. Vídeo de demonstração 13
11. Conclusão e trabalhos futuros 13
12. Referências bibliográficas 13

1. Enquadramento

Este relatório documenta o Trabalho Prático I da unidade curricular de Integração de Sistemas de Informação. O trabalho foi realizado individualmente.

- **Unidade curricular:** Integração de Sistemas de Informação (ISI)
- **Curso:** Licenciatura em Engenharia de Sistemas Informáticos
- **Instituição:** UPCA - Universidade Politécnica do Cávado e do Ave
- **Autor:** André Daniel Freitas Ferreira, n.º 20764
- **Ano letivo:** 2026/2027

O enunciado [1] propõe a aplicação de processos de ETL (*Extract, Transform and Load*) à integração de sistemas de informação ao nível dos dados: recolher dados de fontes e formatos diversos, transformá-los segundo regras de negócio e encaminhá-los para outros sistemas, como bases de dados, ficheiros de exportação, alertas e dashboards. Estes processos são a base da preparação de dados para análise e apoio à decisão [2]. O tema escolhido foi a integração dos dados de vendas de uma loja de informática fictícia, a TechStore.

2. Problema e objetivos

A TechStore é uma loja fictícia de produtos informáticos que vende computadores, monitores, periféricos, equipamentos de armazenamento e acessórios. A informação de que a loja precisa para acompanhar o negócio está distribuída por três ficheiros com formatos diferentes: as vendas num ficheiro CSV, o catálogo de produtos num ficheiro JSON e as categorias num ficheiro XML.

Para além de estarem separados, os dados têm problemas de qualidade: linhas duplicadas, códigos de produto escritos de formas diferentes, campos vazios, quantidades e preços inválidos, datas impossíveis e vendas de produtos que não existem no catálogo. Nestas condições não é possível consultar a informação de forma centralizada nem confiar nos totais de faturação.

O objetivo do trabalho foi desenvolver uma solução de integração que:

- extrai os dados das três fontes heterogéneas;
- limpa, normaliza e valida as vendas, guardando os registos rejeitados e o motivo da rejeição;
- integra vendas, produtos e categorias e calcula a faturação;
- carrega os dados tratados numa base de dados PostgreSQL;
- monitoriza os dados através de um sistema de alertas na própria base de dados;
- apresenta os indicadores e os alertas num dashboard interativo.

3. Estratégia utilizada

O trabalho segue as três fases obrigatórias do enunciado [1]: extração (E), transformação (T) e carregamento (L). A estas fases juntaram-se a monitorização dos dados, através de alertas, e a visualização dos resultados num dashboard. Optou-se por desenvolver scripts próprios em Python, em vez de uma ferramenta gráfica de ETL, porque permitem descrever cada regra de forma explícita, testá-la automaticamente e repetir todo o processo a partir da linha de comandos.

A integração dos três formatos assenta nas chaves comuns às fontes: cada venda do ficheiro CSV refere um produto pelo codigo_produto e cada produto do ficheiro JSON refere uma categoria do ficheiro XML pelo codigo_categoria. O trabalho foi repartido pelas tecnologias da seguinte forma:

- **Python e pandas (extração e transformação)** - o CSV é lido com pandas [3] e o JSON e o XML com os módulos json e xml.etree.ElementTree da biblioteca padrão do Python. Todas as transformações (remoção de duplicados, normalização com expressões regulares, validação, joins, cálculos em Decimal e agregações) são feitas em Python, com pandas e com o módulo re [4].
- **PostgreSQL (carregamento e monitorização)** - os dados tratados são carregados numa base de dados PostgreSQL [5], onde as regras de alertas estão implementadas em PL/pgSQL. A ligação a partir do Python usa o SQLAlchemy e o psycopg, com a configuração lida de um ficheiro .env.
- **Streamlit e Plotly (visualização)** - o dashboard lê os dados e os alertas do PostgreSQL e apresenta-os em gráficos interativos [6], [7].
- **Orquestração** - o script etl.py encadeia as fases e, depois da carga, chama a verificação de alertas. A execução periódica dessa verificação ficou preparada para o Agendador de Tarefas do Windows [8], mas não foi instalada.
- **Testes** - cada fase é verificada com testes automáticos escritos em pytest.

Na entrega, os componentes de integração (etl.py, executar_alertas.py, scripts SQL e scripts do job) estão na pasta dataintegration, os módulos Python, o dashboard e os testes na pasta src, as fontes de dados na pasta data/input e os resultados do ETL na pasta data/output.

3.1 Tecnologias utilizadas

A solução final foi construída com ferramentas de utilização corrente, escolhidas para manter o projeto simples de executar e de explicar.

Tecnologia	Utilização no projeto
Python 3.11 e pandas	Extração, transformação, validação, integração e agregação dos dados
PostgreSQL 18 (SQL e PL/pgSQL)	Armazenamento dos dados tratados, vistas e regras de alertas
SQLAlchemy e psycopg 3	Ligação do Python ao PostgreSQL
python-dotenv	Leitura da configuração da base de dados a partir de um ficheiro .env
Streamlit e Plotly	Dashboard web com gráficos interativos
pytest	Testes automáticos
Agendador de Tarefas do Windows	Execução periódica da verificação de alertas (preparada, não instalada)

Tabela 1 - Tecnologias utilizadas

3.2 Fontes de dados

Os dados são fictícios e foram gerados com uma seed fixa, para que os resultados sejam reproduzíveis. Foram introduzidos de propósito alguns erros, para demonstrar as validações do processo de integração.

Ficheiro	Formato	Conteúdo	Registos
vendas.csv	CSV, separador ;	id_venda, data_venda, codigo_produto, quantidade, preco_unitario	526
produtos.json	Array JSON	codigo_produto, nome_produto, codigo_categoria	30
categorias.xml	XML	codigo_categoria, nome_categoria	5

Tabela 2 - Fontes de dados

As três fontes estão relacionadas: cada venda refere um produto pelo código e cada produto refere uma categoria. Esta relação é a base da integração feita no processo ETL.

4. Transformações: processo ETL em Python

O processo ETL está organizado em módulos Python (extract, transform, load e pipeline) e é executado com o script dataintegration/etl.py. Todas as transformações são feitas em Python com pandas; a base de dados recebe apenas os dados já tratados.

A Figura 1 resume o fluxo das transformações: as três fontes são extraídas, as vendas passam por cinco operações encadeadas e os resultados são carregados em ficheiros CSV e no PostgreSQL. Os registos que não cumprem as regras são retirados do fluxo principal e guardados com o motivo, o que permite reconciliar todas as vendas lidas.

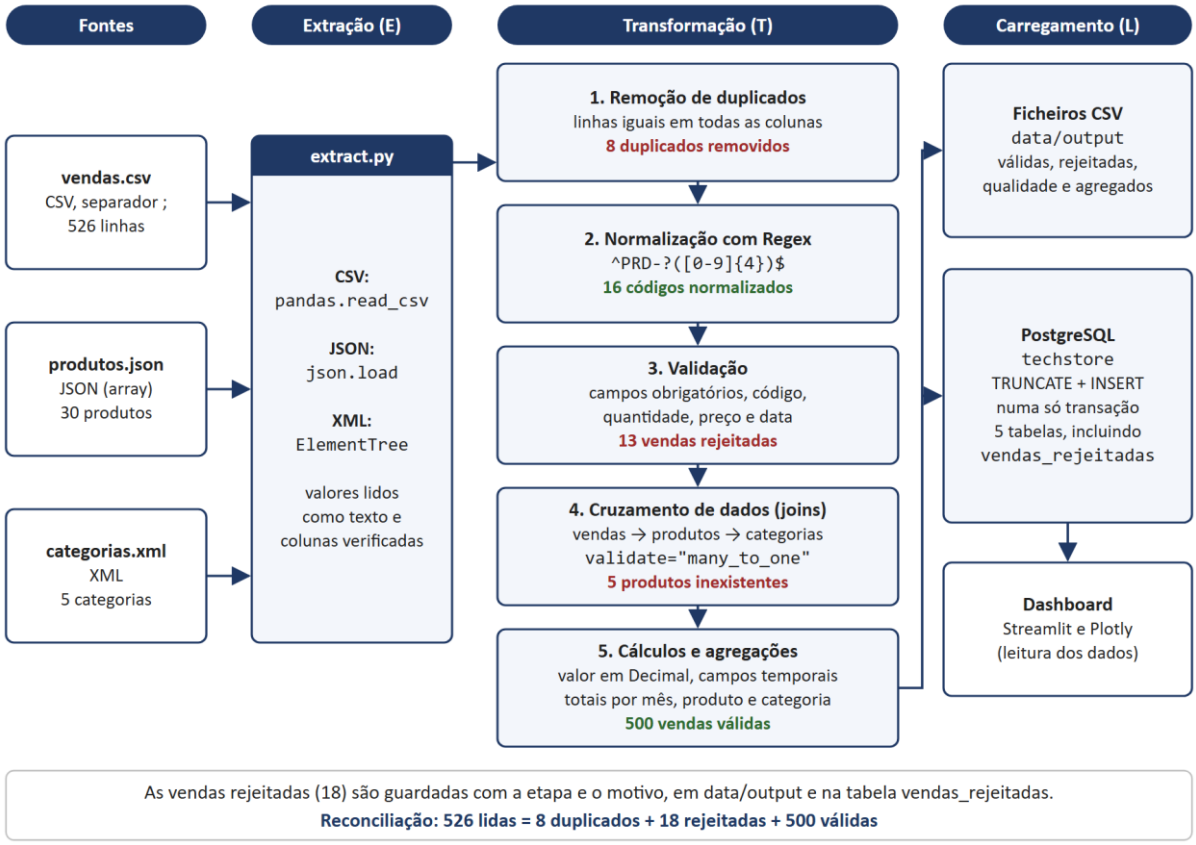


Figura 1 - Diagrama das transformações ETL

4.1 Extração

Os três ficheiros são lidos com caminhos relativos à pasta do projeto: o CSV com pandas, o JSON com a biblioteca json e o XML com ElementTree. Os valores das vendas são lidos como texto, sem conversões automáticas, para que a validação fique concentrada numa só fase.

4.2 Limpeza, normalização e validação

Primeiro são removidas as linhas exatamente iguais em todas as colunas. Depois, os códigos de produto são normalizados com a expressão regular `^PRD-?([0-9]{4})$`: passam a maiúsculas e recebem o hífen em falta (por exemplo, prd-0002 e PRD0003 passam a PRD-0002 e PRD-0003). A seguir, cada venda é validada pelas regras abaixo, por esta ordem; o primeiro problema encontrado define o motivo de rejeição.

Ordem	Regra de validação	Exemplo rejeitado
1	Campos obrigatórios preenchidos	data em falta
2	Código de produto no formato ^PRD-[0-9]{4}$	XYZ
3	Quantidade inteira e positiva	0
4	Preço positivo com, no máximo, duas casas decimais	-129.90
5	Data no formato AAAA-MM-DD e existente no calendário	2026-02-30

Tabela 3 - Regras de validação das vendas

As vendas rejeitadas não são descartadas: ficam guardadas com os valores originais, a etapa em que foram rejeitadas e o motivo.

4.3 Integração e cálculos

As vendas válidas são associadas aos produtos pelo código do produto e os produtos às categorias pelo código da categoria. Os joins usam a opção `validate="many_to_one"`, que impede a multiplicação acidental de vendas caso o catálogo tivesse códigos repetidos. As vendas cujo produto não existe no catálogo são rejeitadas com o motivo "produto inexistente".

O valor de cada venda é calculado como quantidade × preço unitário com o tipo Decimal e arredondado ao cêntimo, evitando os erros de arredondamento dos números de vírgula flutuante. São também criados campos para análise temporal (ano, mês, trimestre e dia da semana) e agregações por mês, por produto e por categoria.

4.4 Carregamento

Os resultados são exportados para ficheiros CSV (pasta data/output) e, com a opção --bd, carregados no PostgreSQL. A carga é feita numa única transação: as tabelas são esvaziadas e preenchidas de novo, pelo que pode ser repetida sem duplicar dados e, em caso de erro, os dados anteriores mantêm-se. No fim, o processo confirma a reconciliação das contagens.

Indicador	Resultado
Vendas lidas do CSV	526
Duplicados exatos removidos	8
Códigos de produto normalizados	16
Rejeitadas na validação	13 (3 campos vazios, 4 quantidades, 3 preços, 3 datas)
Rejeitadas na integração (produto inexistente)	5
Total de vendas rejeitadas	18
Vendas válidas	500
Unidades vendidas	725
Faturação total	69 981,01 €
Reconciliação	526 = 8 duplicados + 18 rejeitadas + 500 válidas

Tabela 4 - Resultados do processo ETL

5. Estrutura da base de dados PostgreSQL

A base de dados techstore guarda os dados tratados num modelo relacional simples. As categorias relacionam-se com os produtos (uma categoria tem vários produtos) e os produtos com as vendas (um produto tem várias vendas).

Tabela	Conteúdo	Registos
categorias	Código e nome de cada categoria	5
produtos	Código, nome e categoria de cada produto (chave estrangeira para categorias)	30
vendas	Vendas válidas, com campos temporais, quantidade, preço e valor total (chave estrangeira para produtos)	500
vendas_rejeitadas	Valores originais, etapa e motivo de cada venda rejeitada	18
resumo_qualidade	Indicadores de qualidade da última carga	20
alertas	Ocorrências detetadas pelas regras de alertas (capítulo 6)	21
alertas_execucoes	Registo de cada verificação de alertas	20

Tabela 5 - Tabelas da base de dados (registos na data do relatório)

A integridade é garantida também pela própria base de dados, através de chaves primárias e estrangeiras e de restrições CHECK: formato do código de produto, quantidade e preço positivos e valor_total igual a quantidade × preco_unitario. Os valores monetários usam o tipo NUMERIC. Existem ainda quatro vistas de apoio à análise: vw_vendas_detalhe, vw_faturacao_mensal, vw_vendas_produto e vw_vendas_categoria.

As tabelas e as vistas pertencem ao administrador da base de dados. A aplicação usa um utilizador próprio (techstore_app) que pode ler e carregar dados, mas não pode criar nem alterar a estrutura.

6. Sistema de alertas

O processo ETL garante que os dados carregados são válidos, mas não indica se a qualidade das fontes está a piorar nem se o negócio tem situações que merecem atenção. Para isso foi criado um sistema

de alertas na própria base de dados: um conjunto de regras de negócio que identifica estas situações e guarda as ocorrências com o respetivo histórico. Os alertas podem ser consultados diretamente no PostgreSQL, sem depender do dashboard.

6.1 Regras de negócio

Regra	Critério	Alertas reais
A1 - Qualidade dos dados	Vendas rejeitadas acima de 3% das vendas lidas (os duplicados não contam como rejeições)	1 (3,42%)
A2 - Queda de faturação	Queda superior a 30% entre dois meses completos consecutivos	0 (maior queda: 29,8%)
A3 - Produtos sem vendas	Produtos sem vendas nos últimos 60 dias até à última data de venda	2
A4 - Vendas atípicas	Valor acima de Q3 + 1,5 × IQR da respetiva categoria	17
A5 - Concentração de faturação	Uma categoria com mais de 50% da faturação de um mês completo	1 (Computadores, abril: 58,7%)

Tabela 6 - Regras de alertas e resultados com os dados reais

Os limites foram confirmados com os dados reais e não foram ajustados para forçar alertas. Por exemplo, a regra A2 não gera alertas porque a maior queda mensal é de 29,8%; o seu funcionamento foi verificado nos testes com dados controlados. Na regra A4 foi usado o critério do intervalo interquartil [9], aplicado por categoria porque, calculado sobre todas as vendas, marcaria quase todas as vendas de computadores, que são caras mas normais. Uma venda atípica é apenas invulgar, não é inválida.

6.2 Funções SQL

As regras estão implementadas na função PL/pgSQL verificar_alertas(). Em cada execução, a função avalia as cinco regras, cria os alertas novos, atualiza os que continuam a verificar-se e resolve automaticamente os que deixaram de se verificar. Cada ocorrência tem uma chave (por exemplo, A3:PRD-0005) e um índice único garante que existe no máximo uma ocorrência em aberto por chave, pelo que execuções repetidas não criam duplicados. Cada execução fica registada na tabela alertas_execucoes.

Todo o processamento decorre dentro de um bloco de tratamento de erros: se algo falhar, as alterações aos alertas são revertidas e a execução fica registada com o estado "erro" e a mensagem correspondente. A função não altera as tabelas de vendas.

Um alerta pode estar ativo, reconhecido ou resolvido. No dashboard é possível reconhecer um alerta (função reconhecer_alerta()), mas só a função de verificação o pode resolver, quando a regra deixa de se verificar. Se a situação voltar a acontecer, é criada uma nova ocorrência e a anterior fica no histórico.

As duas funções pertencem a um role sem acesso de sessão (techstore_alertas), com as permissões mínimas necessárias. O utilizador da aplicação pode apenas ler os alertas e executar as funções; não consegue criar, alterar ou apagar alertas diretamente.

7. Jobs e orquestração

Neste trabalho, um job é uma unidade de execução que encadeia vários passos do processo de integração e que pode ser lançada manualmente ou por um agendador. A Figura 2 mostra os jobs implementados, as formas de os executar e os resultados de cada um. As linhas tracejadas representam o agendamento no Windows, que ficou preparado mas não foi instalado.

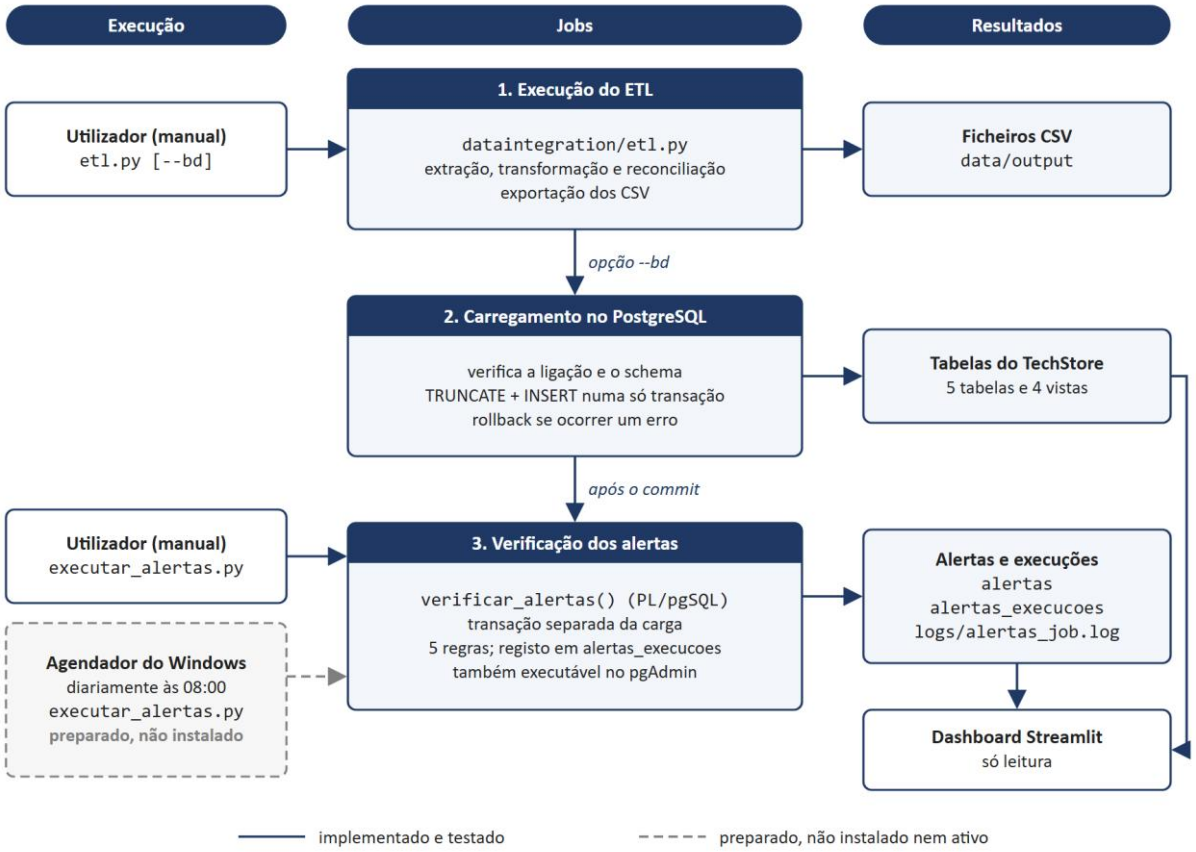


Figura 2 - Diagrama dos jobs: execução do ETL, carregamento, verificação de alertas e agendamento preparado

7.1 Execução do ETL

O comando dataintegration/etl.py executa a extração, a transformação e a exportação dos resultados para a pasta data/output. Antes da exportação, o processo confirma a reconciliação das contagens; se esta falhar, a execução é interrompida e nada é exportado nem carregado.

7.2 Carregamento no PostgreSQL

Com a opção --bd, o mesmo comando lê a configuração do ficheiro .env, testa a ligação e verifica se as tabelas e as vistas existem. A carga substitui os dados numa única transação (TRUNCATE seguido de INSERT): se ocorrer um erro, a transação é revertida e os dados anteriores mantêm-se.

7.3 Verificação dos alertas

Depois de confirmada a carga, o ETL chama a função verificar_alertas() numa transação separada, pelo que uma falha na verificação não desfaz a carga. Importa distinguir três componentes:

- **Função SQL** - verificar_alertas(), na base de dados, onde estão todas as regras. Está instalada e testada e pode ser executada, por exemplo, no pgAdmin.

- **Script Python** - executar_alertas.py, que liga à base de dados, chama a função e regista o resultado. É também chamado automaticamente após cada carga do ETL.
- **Job agendado** - uma tarefa do Agendador de Tarefas do Windows que executaria o script todos os dias às 08:00.

7.4 Agendamento no Windows (preparado, não instalado)

O job **está preparado mas não foi instalado nem está ativo**. A tarefa executaria o script executar_alertas.py com a origem "agendador". Os scripts para a criar, verificar e remover estão na pasta dataintegration/agendamento e a configuração está documentada (utilizador atual sem privilégios elevados, sem password guardada, execução assim que possível se o computador estiver desligado à hora prevista). A criação da tarefa é uma alteração permanente ao Windows e não foi feita. Atualmente, os alertas são verificados após cada carga do ETL ou manualmente. Foram consideradas alternativas dentro do PostgreSQL, mas o pg_cron não tem versão oficial para Windows e o pgAgent exigiria instalar software e um serviço adicional.

8. Dashboard Streamlit

O dashboard foi desenvolvido em Streamlit, com gráficos Plotly, e tem duas páginas. A página Análise de Vendas lê os dados do PostgreSQL e apresenta os indicadores principais, a evolução da faturação mensal, os cinco produtos mais vendidos e a faturação e o número de vendas por categoria, com filtros por intervalo de datas, categoria e produto (Figura 3).



Figura 3 - Página Análise de Vendas: indicadores, faturação mensal e top 5 de produtos

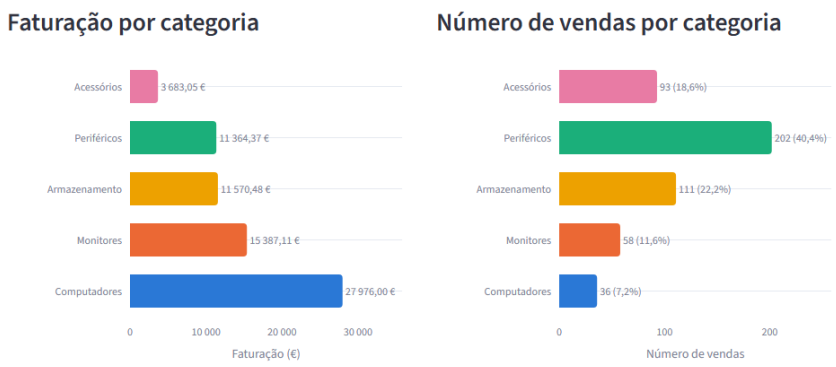


Figura 4 - Faturação e número de vendas por categoria

Os gráficos por categoria (Figura 4) mostram que as categorias com mais vendas não são as que mais faturam: os Periféricos têm 202 vendas, mas os Computadores, com 36 vendas, representam a maior faturação (27 976,00 €). A página inclui também tabelas com as vendas detalhadas (Figura 5), os registos rejeitados e o resumo da qualidade dos dados. As agregações usam as mesmas funções do ETL, para que as regras de cálculo existam num único sítio. Sem base de dados, o dashboard funciona em modo de demonstração com os ficheiros CSV exportados pelo ETL.



Venda	Data	Produto	Categoria	Qtd.	Preço unitário	Valor total	Código lido	Código normalizado
V00001	01/01/2026	SSD NVMe 1TB	Armazenamento	1	71.91 €	71.91 €	PRD-0021	PRD-0021
V00002	01/01/2026	Conjunto Teclado e Rato Sem Fios	Periféricos	1	39.90 €	39.90 €	PRD-0013	PRD-0013
V00003	03/01/2026	Cabo HDMI 2.1 2m	Acessórios	3	12.90 €	38.70 €	PRD-0030	PRD-0030
V00004	03/01/2026	Hub USB-C 7 em 1	Acessórios	2	31.41 €	62.82 €	PRD-0027	PRD-0027
V00005	03/01/2026	Rato Ótico USB	Periféricos	4	9.90 €	39.60 €	PRD-0014	PRD-0014
V00006	04/01/2026	Disco Externo 1TB USB 3.0	Armazenamento	1	59.90 €	59.90 €	PRD-0023	PRD-0023
V00007	04/01/2026	SSD NVMe 2TB	Armazenamento	1	125.91 €	125.91 €	PRD-0022	PRD-0022
V00008	04/01/2026	SSD Externo Portátil 1TB	Armazenamento	1	104.41 €	104.41 €	PRD-0025	PRD-0025
V00009	04/01/2026	Conjunto Teclado e Rato Sem Fios	Periféricos	3	39.90 €	119.70 €	PRD-0013	PRD-0013
V00010	07/01/2026	Conjunto Teclado e Rato Sem Fios	Periféricos	1	39.90 €	39.90 €	PRD-0013	PRD-0013

Figura 5 - Tabela de vendas detalhadas, com o código lido e o código normalizado

A página Centro de Alertas consulta os alertas gerados na base de dados. Mostra a última execução da verificação, o número de alertas por estado, por severidade e por tipo, e a lista de alertas com filtros (Figura 6). Esta página não calcula alertas: apenas os apresenta.



Figura 6 - Centro de Alertas: última verificação e alertas em aberto

Cada alerta da lista (Figura 7) pode ser aberto para ver a descrição e os valores que o justificam, por exemplo o número de vendas rejeitadas e a taxa de rejeição no alerta A1, e pode ser reconhecido. Existem também separadores com o histórico de alertas resolvidos e com as execuções da verificação.

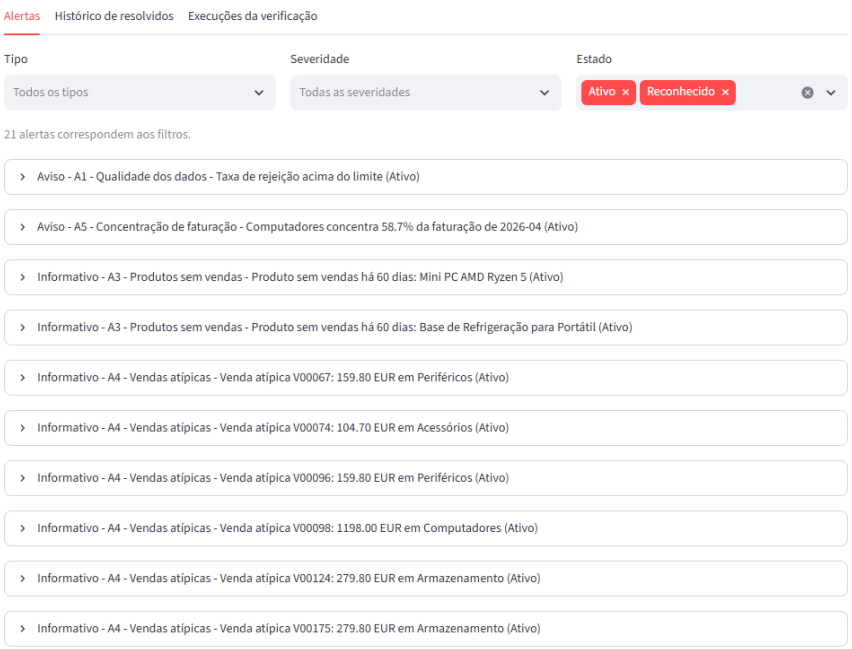


Figura 7 - Lista de alertas com filtros por tipo, severidade e estado

9. Testes e resultados

O projeto tem 98 testes automáticos, escritos com pytest. Sempre que possível, os testes usam pequenos conjuntos de dados criados no próprio teste, em vez de dependerem apenas dos ficheiros originais. Os testes que precisam do PostgreSQL só são executados quando a base de dados está configurada; os testes de alertas correm dentro de uma transação que é sempre revertida, pelo que não alteram os dados nem os alertas reais.

Ficheiro de testes	Testes	O que verifica
test_extract.py	4	Leitura das três fontes
test_transform.py	45	Duplicados, Regex, validações, joins, cálculo em Decimal e agregações
test_pipeline.py	10	Processo completo, reconciliação e integridade dos resultados
test_load.py	8	Exportação para CSV e carga no PostgreSQL (repetição, totais e rollback)
test_alertas.py	31	Cinco regras, limites exatos, falta de dados, duplicados, resolução, reabertura, erros, permissões e dashboard

Tabela 7 - Testes automáticos

Na última execução completa, com o PostgreSQL disponível, os 98 testes foram aprovados. A mesma execução foi repetida de forma independente sobre a pasta de entrega final, com um ambiente Python novo, com o mesmo resultado.

Na estrutura de entrega (pasta tp01-20764), o ETL foi novamente executado com os mesmos resultados e os testes que não dependem da base de dados terminaram com 65 aprovados e 33 ignorados. A ligação ao PostgreSQL e as duas páginas do dashboard foram verificadas em modo de leitura.

A primeira verificação real dos alertas, executada manualmente, gerou 21 alertas: 1 de qualidade dos dados, 2 de produtos sem vendas, 17 de vendas atípicas e 1 de concentração de faturação. Uma segunda verificação, feita automaticamente após uma nova carga do ETL com os mesmos dados,

terminou com 0 alertas novos, 21 atualizados e 0 resolvidos, o que confirma que a repetição não cria duplicados.

10. Vídeo de demonstração

O vídeo de demonstração, com cerca de 5 minutos, mostra o funcionamento da solução: a execução do processo ETL, a integração com o PostgreSQL, o dashboard e o sistema de alertas. O vídeo pode ser aberto através da ligação abaixo ou do QR Code da Figura 8.

Ligação: https://youtu.be/N6skA5iyNMo



Figura 8 - QR Code de acesso ao vídeo de demonstração

11. Conclusão e trabalhos futuros

O trabalho permitiu integrar três fontes de dados heterogéneas (CSV, JSON e XML) num processo ETL completo, desde a extração até ao armazenamento numa base de dados relacional e à apresentação num dashboard. Das 526 vendas originais, foram removidos 8 duplicados e rejeitadas 18 vendas com o motivo identificado, ficando 500 vendas válidas, com 725 unidades vendidas e 69 981,01 € de faturação, ligadas a 30 produtos e 5 categorias.

O sistema de alertas acrescentou monitorização e lógica de negócio na própria base de dados: cinco regras SQL geraram 21 alertas reais, guardados com histórico e consultáveis no PostgreSQL e no Centro de Alertas. A verificação é executada após cada carga do ETL ou manualmente; o job automático ficou preparado e documentado, mas não foi instalado.

Como trabalhos futuros, destacam-se a instalação do job agendado, a carga incremental em vez da substituição completa das tabelas, o registo de vários motivos por venda rejeitada e a melhoria da formatação dos textos dos alertas (que usam ponto decimal). Com dados reais seriam também necessárias validações adicionais, como preços de referência por produto.

12. Referências bibliográficas

As referências web foram consultadas em outubro de 2026.

[1] L. Ferreira e Ó. Ribeiro, *Integração de Sistemas de Informação - Trabalho Prático I* (enunciado), Licenciatura em Engenharia de Sistemas Informáticos, ano letivo 2026/27.

[2] R. Kimball e J. Caserta, *The Data Warehouse ETL Toolkit: Practical Techniques for Extracting, Cleaning, Conforming, and Delivering Data*. Indianapolis, IN: Wiley, 2004.

[3] The pandas development team, "pandas documentation". Disponível em: https://pandas.pydata.org/docs/

[4] Python Software Foundation, "re - Regular expression operations", *Python 3.11 documentation*. Disponível em: https://docs.python.org/3.11/library/re.html

[5] The PostgreSQL Global Development Group, "PostgreSQL 18 Documentation". Disponível em: https://www.postgresql.org/docs/18/

[6] Streamlit, "Streamlit documentation". Disponível em: https://docs.streamlit.io/

[7] Plotly, "Open Source Python Graphing & Data Visualization Library". Disponível em: https://plotly.com/python/

[8] Microsoft, "Task Scheduler for developers", *Microsoft Learn*. Disponível em: https://learn.microsoft.com/en-us/windows/win32/taskschd/task-scheduler-start-page

[9] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA: Addison-Wesley, 1977.